

4주차 - Transformer, BERT

자료

https://www.canva.com/design/DAHAFUht3GE/BnlsQAb1Jc30mCQh0oLiNA/view?utm_content=DAHAFUht3GE&utm_campaign=designshare&utm_medium=link2&utm_source=uniquelinks&utlId=h11

1. Seq2Seq

정답 문장을 하나씩 알려주며(teacher forcing) 다음 단어 예측하도록 학습함

RNN 기반

2. Attention

1. 디코더의 현재 상태와 각 인코더 상태 간의 유사도를 계산

$$\text{score}_{t,i} = s_t^\top h_i$$

- decoder 상태 s_t
- encoder 상태 h_i

2. Attention Weight

score에 softmax를 적용해 중요도 비율을 구한다.

3. Attention Value (Context Vector)

중요도를 가중치로 하여 인코더 상태들을 가중합해 입력 정보를 요약한다.

$$\text{Value}_t = \sum_i \alpha_{t,i} h_i$$

- encoder hidden state들을 중요한 것만 많이 섞은 벡터

4. 출력 예측

Attention value와 디코더 상태를 결합해 다음 단어를 예측한다.

3. Transformer

RNN 없이 Encoder가 입력을 해석하고 Decoder가 이를 참고해 출력을 생성하는 Attention 기반 모델

- RNN 제거 → 병렬 처리 가능
- Attention만으로 문맥 파악
- 긴 문장, 장기 의존성에 강함

self-attention: 각 단어는 문장 내 모든 단어를 참고해 문맥이 반영된 새로운 의미 벡터로 변환

4. Encoder

입력 임베딩

- Q, K, V 생성
- (각 head별) Scaled Dot-Product Attention
- head 결과들 concat
- Encoder 출력

- **Q (Query):** “지금 이 단어가 무엇을 찾고 있는지”
- **K (Key):** “각 단어의 특징 요약”
- **V (Value):** “실제로 가져올 의미 정보”

하나의 Attention은 **한 종류의 관계만** 잘 봄
 각 head마다 다른 가중치 사용

5. Decoder

- **Masked Multi-Head Attention**
 : 미래 토큰을 차단한 상태로 이전 출력만 참고
- **Position-wise FFN**
 : 각 토큰을 독립적으로 비선형 변환
- **Residual + LayerNorm**
 : 깊은 모델의 안정적 학습을 위한 구조

6. BERT

- Transformer **Encoder만** 사용
 - Self-Attention에서 **미래 토큰을 가리지 않음**
 - 문장 전체를 동시에 보고 각 토큰의 의미를 계산
- BERT의 목적은 생성이 아니라 이해

1. Pre-training 단계

1) Masked Language Model (MLM)

- 문장 중 일부 토큰을 [MASK]로 가림
- 좌·우 문맥을 모두 이용해 해당 토큰 예측 → **양방향 문맥 활용이 가능**

2) Next Sentence Prediction (NSP)

- 문장 A와 B가 실제로 이어지는 문장인지 판단
- 문장 간 관계 학습

2. Fine-tuning 단계

- Pretrained BERT 위에
- **Task-specific head 하나만 추가**
- 문장 분류: [CLS] 토큰 → classifier
- QA: 시작/끝 위치 예측
- NER: 토큰 단위 분류

Input Form	Task	핵심 출력
Sentence A + B	문장 쌍 분류	[CLS]
Single Sentence	문장 분류	[CLS]
Question + Paragraph	질의응답	모든 토큰
Single Sentence	토큰 분류	각 토큰

3. BERT 입력 구조

BERT는 항상 **하나의 토큰 시퀀스**를 입력으로 받는다.

[CLS] Sentence A [SEP] Sentence B [SEP]

- [CLS]: 전체 문장 대표 토큰 (분류용)
- [SEP]: 문장 구분 토큰
- Sentence A/B: NSP를 위한 두 문장

4. BERT의 Embedding 구성

각 토큰의 최종 입력 임베딩은 **세 가지의**

$E_i = E_{\text{token}} + E_{\text{segment}} + E_{\text{position}}$

1) Token Embedding

단어의 의미, 3만개의 토큰사전으로 이루어진 WordPiece embedding 사용

2) Segment Embedding

이 토큰이 Sentence A인지 B인지 표시, NSP 학습을 위해 필요

3) Position Embedding

토큰의 상대적 순서 정보

- **GPT**: 문장을 **생성**하는 데 특화 (Decoder-only)
- **BART**: **이해 + 생성**을 둘 다 잘함 (Encoder-Decoder)